ANÁLISIS DE ALGORITMOS

Guía Práctica

Contenido Sinóptico

Análisis de la complejidad en tiempo de algoritmos iterativos y recursivos. Análisis de la complejidad en espacio de algoritmos iterativos.

Calcular la complejidad en tiempo del peor caso de cada uno de los siguientes algoritmos iterativos. Utilice la notación *O*-grande para expresar los resultados.

(Nota: Se tomará en cuenta que las constantes N, M y los tipos vector = **arreglo** [1..N] **de entero** y matriz = **arreglo** [1..N][1..M] **de entero** están definidos)

1. Swap de dos números

```
01  proc swap(ref real: a, b)
02  var
03        real: aux
04  inicio
05        aux ← a
06        a ← b
07        b ← aux
08  fproc
```

2. Recorrido de un vector

```
01  proc imprimirVector(vector: V; entero: n)
02  var
03        entero: i
04  inicio
05        para i ← 1 hasta n hacer
06              escribir(V[i])
07        fpara
08  fproc
```

3. Recorrido de un vector de dos en dos

```
01  proc imprimirVectorDosEnDos (vector: V; entero: n)
02  var
03        entero: i
04  inicio
05        para i ← 1 hasta n en 2 hacer
06              escribir(V[i])
07        fpara
08  fproc
```


impulsado por CamScanner

4. Factorial de *n*

```
01 func factorial(entero: n): entero
02 var
03       entero: fact
04 inicio
05       fact ← 1
06       para i ← 2 hasta n hacer
07             fact ← fact * i
08       fpara
09       retornar(fact)
10 ffunc
```

5. Recorrido de Matrices

```
01 proc imprimirMatriz(matriz: M; entero: n)
02 var
03       entero: i, j
04 inicio
05       para i ← 1 hasta n hacer
06             para j ← 1 hasta n hacer
07                   escribir(M[i, j])
08             fpara
09       fpara
10 fproc
```

6. Procedimiento 1

```
01 proc proc1(entero: n; ref entero: sum)
02 var
03       entero: i, j
04 inicio
05       sum ← 0
06       para i ← 1 hasta n - 1 hacer
07             para j ← 1 hasta 2*n hacer
08                   sum ← sum + 1
09             fpara
10       fpara
11 fproc
```

7. Fibonacci de *n*

```
01 func fibonacci(entero: n): entero
02 var
03       entero: fn, ant, act, i
04 inicio
05       si (n = 0 ∨ n = 1) entonces
06             retornar (n)
07       sino
08             ant ← 0
09             act ← 1
10             para i ← 2 hasta n en 1 hacer
11                   fn ← ant + act
12                   ant ← act
13                   act ← fn
14             fpara
15             retornar(fn)
16       fsi
17 ffunc
```


impulsado por CamScanner

8. Multiplicación de Matrices

```
01  proc matMult(matriz: A, B; ref matriz: C; entero: n)
02  var
03        entero: i, j, k
04  inicio
05        para i ← 1 hasta n hacer
06              para j ← 1 hasta n hacer
07                    C[i, j] ← 0
08                    para k ← 1 hasta n hacer
09                          C[i, j] ← C[i, j] + A[i, k]*B[k, j]
10                    fpara
11              fpara
12        fpara
13  fproc
```

9. Combinatoria de *n* en *m* (tome en cuenta el resultado obtenido en el análisis del factorial de *n*)

```
01  func combinatoria_nm(entero: n, m): entero
02  var
03        entero: p, a, b, c
04  inicio
05        si(n < m) entonces
06              retornar(0)
07        sino
08              p ← n - m
09              a ← factorial(n)
10              b ← factorial(m)
11              c ← factorial(p)
12              retornar(a div (b*c))
13        fsi
14  ffunc
```

10. Búsqueda lineal en un vector

```
01  func busquedaLineal(vector: V; entero: n; entero: e): entero
02  var
03        entero: i
04        lógico: encontrado
05  inicio
06        i ← 1
07        encontrado ← falso
08        mientras(¬encontrado ∧ i ≤ n) hacer
09              si(V[i] = e) entonces
10                    encontrado ← verdadero
11              sino
12                    i ← i + 1
13              fsi
14        fmientras
15        si(encontrado) entonces
16              retornar(i)
17        sino
18              retornar(-1) // Una posición inválida
19        fsi
20  ffunc
```


impulsado por CamScanner

11. Procedimiento 2

```
01  proc proc2(entero: n)
02  var
03        entero: i, j, k
04  inicio
05        para i ← 1 hasta n - 1 hacer
06              para j ← i + 1 hasta n hacer
07                    para k ← 1 hasta j hacer
08                          escribir(i, " ", j, " ", k)
09                    fpara
10              fpara
11        fpara
12  fproc
```

12. Búsqueda Binaria

```
01  func busquedaBinaria(vector: V; entero: n; entero: e): entero
02  var
03        entero: inferior, superior, mitad
04        lógico: encontrado
05  inicio
06        encontrado ← falso
07        inferior ← 1
08        superior ← n
09        mientras(¬encontrado ∧ inferior ≤ superior) hacer
10              mitad ← (inferior + superior) div 2
11              si(v[mitad] = e) entonces
12                    encontrado ← verdadero
13                    pos ← mitad
14              sino
15                    si(v[mitad] < e) entonces
16                          inferior ← mitad + 1
17                    sino
18                          superior ← mitad - 1
19                    fsi
20              fsi
21        fmientras
22        si(encontrado) entonces
23              retornar(pos)
24        sino
25              retornar(-1)
26        fsi
27  ffunc
```

Calcular la complejidad en tiempo del peor caso de cada uno de los siguientes algoritmos recursivos. Utilice la notación *O*-grande para expresar los resultados.

(Nota: Se tomará en cuenta que las constantes N, M y los tipos vector = **arreglo** [1..N] **de entero** y matriz = **arreglo** [1..N][1..M] **de entero** están definidos)

13. Factorial

```
01  func factorial(entero: n): entero
02  inicio
03        si(n = 0 ∨ n = 1) entonces
04              retornar(1)
05        sino
06              retornar(n * factorial(n-1))
07        fsi
08  ffunc
```


impulsado por CamScanner

14. Fibonacci

```
01 func fibonacci(entero: n): entero
02 inicio
03       si(n = 0 ∨ n = 1) entonces
04             retornar(n)
05       sino
06             retornar(fibonacci(n-1) + fibonacci(n-2))
07       fsi
08 ffunc
```

15. Potencia

```
01 func potencia(real: a, entero: b): real
02 inicio
03       si(a = 0) entonces
04             retornar(0)
05       sino
06             si(b = 0) entonces
07                   retornar(1)
08             sino
09                   retornar(a * potencia(a, b - 1))
10             fsi
11       fsi
12 ffunc
```

16. Binario a Decimal

```
01 func binarioADecimal(entero: bin, pot): entero
02 inicio
03       si(bin < 10) entonces
04             retornar(bin*(2**pot))
05       sino
06             retornar(binarioADecimal(bin div 10, pot + 1) + (bin mod 10)*(2**pot))
07       fsi
08 ffunc
```

17. Búsqueda binaria

```
01 func busquedaBinaria(vector: a, entero: x, ini, fin): lógico
02 var
03       entero: mitad
04 inicio
05       si(ini > fin) entonces
06             retornar(falso)
07       sino
08             mitad = (ini+fin) div 2
09             si(a[mitad] = x) entonces
10                   retornar(verdadero)
11             sino
12                   si(x > a[mitad]) entonces
13                         retornar(busquedaBinaria(a, x, mitad+1, fin))
14                   sino
15                         retornar(busquedaBinaria(a, x, ini, mitad-1))
16                   fsi
17             fsi
18       fsi
19 ffunc
```


impulsado por CamScanner

18. Compare los resultados de las versiones iterativa y recursiva de los algoritmos factorial, fibonacci y búsqueda binaria. En cada uno de los casos responda:

a) ¿Cuál de las versiones presenta mejor desempeño?

b) ¿Por qué?

Complejidad en Espacio

19. Calcular la complejidad en espacio de cada uno de los algoritmos anteriores.

impulsado por CamScanner